

BookishApp

Rapport Technique

Next.js · Prisma (Neon) · Stack Auth

Février 2025

Alexandrine Dubé

Choix de l'architecture

Pour ce projet web full-stack, j'ai décidé de construire l'application entièrement avec Next.js. Bien que cette technologie ait été introduite plus tard dans la session, le défi de migrer vers une architecture qui n'a pas de séparation explicite du backend et du frontend, comme Node, m'intéressait énormément. Cette décision, bien que complexe à implémenter au départ, a grandement simplifié le déploiement par la suite, sans avoir à gérer le déploiement et la connexion du backend et du frontend.

Structure des routes

Le service web est structuré grâce à l'App Router de Next.js, qui utilise la structure des dossiers sous app pour définir automatiquement les routes. J'ai tout de même séparé mon projet en quelques sections logiques. J'ai le groupe de route (auth) qui contient les pages de connexion et d'inscription qui sont gérées par Stack Auth. Par la suite, j'ai le groupe (protected) qui contient le groupe de pages qui est seulement accessible aux utilisateurs connectés : les étagères, le dashboard, et la page des critiques. La protection des routes dans ce groupe est simplement gérée dans le fichier layout à la racine du groupe.

Toute la logique de base de données est mise à part sous app/actions, avec un fichier par ressource. Ces fichiers contiennent les Server Actions qui s'exécutent du côté serveur et qui accèdent donc à Prisma et à la base de données. Les composants React quant à eux sont dans le dossier app/components/ avec quelques sous-dossiers pour retrouver, par exemple, les composants reliés aux formulaires, facilement. Finalement, le dossier /lib regroupe différents fichiers utilitaires, comme la configuration de Prisma et de Stack Auth, ainsi que sync-user pour synchroniser Stack Auth avec ma base de données.

Mon plus grand apprentissage dans ce projet est la gestion et l'implémentation des Server Components et des Client Components. Je devais bien réfléchir avant de commencer pour garder en tête que les Server Components peuvent appeler Prisma, mais ne peuvent pas utiliser useState/useEffect. Alors que les Client Components peuvent être interactifs, ils s'exécutent dans le navigateur mais ne peuvent pas importer Prisma sous peine de causer une erreur. Donc, souvent, mes Server Components récupèrent des données et les passent en props au Client Component qui en a besoin.

Authentification

Pour l'authentification, j'ai choisi Stack Auth. Ce choix s'explique d'abord par le fait qu'il était conseillé pour les projets React/Next.js, mais aussi parce que je me sentais déjà relativement à l'aise avec Clerk et que je souhaitais explorer une alternative moins habituelle pour moi.

La gestion des accès est différenciée selon le statut de l'utilisateur. Pour les visiteurs non connectés, il est uniquement possible de consulter la page d'accueil et d'explorer les options de recherche pour découvrir les différents livres disponibles. Pour les utilisateurs possédant un compte, trois étagères de base sont automatiquement créées à l'inscription : À lire, En cours et Terminés. Ces trois catégories représentent les étapes fondamentales du service, permettant de classer les livres qu'on souhaite lire, ceux qu'on est en train de lire, et ceux qu'on a terminés. Il est également possible de créer des étagères personnalisées supplémentaires sans restriction.

Dans le tableau de bord, l'utilisateur peut visualiser ses livres en cours de lecture accompagnés d'une barre de progression, ainsi qu'une option pour mettre à jour sa progression à tout moment. Il est aussi possible de se fixer un objectif de lecture annuel : l'application vérifie automatiquement tous les livres dont la date de fin de lecture correspond à l'année en cours pour calculer la progression vers cet objectif.

Chaque carte de livre est cliquable et permet d'accéder à la description complète de l'ouvrage. Il est également possible de consulter les critiques de tous les autres utilisateurs, la moyenne des notes étant calculée et affichée dynamiquement.

La gestion des sessions est entièrement prise en charge par Stack Auth, qui s'intègre nativement avec le système de composants serveur de Next.js, permettant une authentification simple, entre autres avec Google/Gmail, et avec une persistance de la session utilisateur grâce à l'utilisation de cookies pour maintenir la connexion.

Défis rencontrés

Mon plus grand défi a été de bien comprendre l'architecture Next.js, et plus particulièrement le fonctionnement des Server Actions : comment les définir, comment les appeler depuis les composants React, et dans quels contextes ils peuvent ou ne peuvent pas être utilisés. La différence entre composants serveur et composants client, en plus de faire passer mon projet de Node à Next, a pris beaucoup plus d'heures que ce que je m'attendais.

Un autre changement a été l'amélioration du schéma de base de données depuis la remise du frontend du projet. Travailler plus concrètement sur le frontend m'a amenée à

simplifier certaines tables qui s'avéraient plus complexes que nécessaires, afin d'adapter le modèle de données aux besoins réels de l'interface.

Améliorations futures

Avec davantage de temps, il y a plusieurs fonctionnalités que j'aurais aimé implémenter dans mon application (mais je risque de bien m'amuser à les faire pendant notre relâche).

L'intégration de l'API Google Books permettrait aux utilisateurs de rechercher des livres par titre, auteur ou ISBN et de les ajouter à leur bibliothèque sans avoir à saisir manuellement les informations. Ce fonctionnement actuel est bien, mais j'aurais aussi aimé avoir les données dans ma base de données, par exemple avec le Data Dump mensuel d'Open Library, qui aurait permis un peu plus de flexibilité sur la gestion des données de livres et aurait été au final moins complexe qu'avec l'API Google. Par contre, la limite de Go dans Neon m'a obligée à être plus conservative et à plutôt appeler l'API de Google Books.

L'amélioration la plus logique pour BookishApp est bien sûr l'implémentation du côté plus « réseau social » de la chose. L'idée serait de permettre aux utilisateurs de consulter le profil public des autres membres : leurs bibliothèques, leurs critiques, leurs objectifs de lecture annuel, etc., et de les suivre à la manière d'un réseau social, pour ensuite développer un fil d'actualité avec les mises à jour des lectures des utilisateurs que nous suivons. Chaque avancement de lecture (changement de page, passage d'une étagère à une autre, fin d'un livre) générerait une mise à jour visible dans le fil d'actualité. Les autres utilisateurs pourraient alors réagir ou laisser un commentaire sur ces mises à jour, transformant BookishApp d'un simple gestionnaire de bibliothèque en une véritable communauté de lecteurs !

En s'appuyant sur les genres favoris, les notes attribuées et l'historique de lecture de chaque utilisateur, il pourrait être intéressant de créer un moteur de recommandations qui pourrait suggérer des livres susceptibles d'intéresser l'utilisateur, et peut-être même recommander des nouvelles sorties.

Finalement, le développement d'une application mobile avec React Native partagerait la logique métier avec le projet web existant et rendrait le tout très intéressant. De plus, l'ajout d'un scanner de code-barres pour ajouter des livres physiques serait une amélioration intéressante pour les lecteurs aguerris qui ne veulent pas rejoindre une communauté de la sorte en raison de la large quantité de livres qu'ils devraient manuellement entrer dans l'application. Dans le même ordre d'idée, pouvoir importer un fichier Excel, un peu comme Goodreads le permet, pour rapidement importer les livres et lectures déjà faites, permettrait à plus d'utilisateurs de transférer vers BookishApp.